Clasificación Automática de Minerales Mediante Extracción de Características y Aprendizaje Clásico

Maria Fernanda Atencia, Stiven Suárez Marín, Michael Stiven Tabares Tobón

Universidad de Antioquia — Facultad de Ingeniería — Procesamiento Digital de Imágenes — 2026-II

Resumen—Este trabajo presenta un sistema de visión por computador para clasificar cuatro minerales (cuarzo, pirita, malaquita, hematita) a partir de imágenes RGB. Se construyó un banco de 240 imágenes (60 por clase) combinando el dataset público tedqc/mineral-dataset (HuggingFace, CC0) con fotografías propias con celular. El pipeline de preprocesamiento (resize bicúbico 128×128, Gaussiano 5×5 σ=1.0 justificado por PSNR/SSIM de la Parte 1, CLAHE sobre L de LAB, Otsu y limpieza morfológica 3×3) alimenta la extracción de siete familias de descriptores: HOG, color HSV/LAB con momentos estadísticos, banco de Gabor multi-orientación, LBP uniforme, matrices GLCM en 4 direcciones y los 7 momentos invariantes de Hu (1835 dims). Se entrenaron SVM RBF, k-NN, Random Forest (con class_weight='balanced') y MLP, más Voting y Stacking ensembles, bajo seis configuraciones de descriptores E1–E5, con validación cruzada 5-fold y GridSearchCV. El mejor resultado fue Random Forest sobre descriptores de color (HSV/LAB) y textura (GLCM) en 38 dimensiones: 91.9% ± 2.5% en CV-5 y 83.3% en test (F1 macro 0.836), mejorando el recall de las clases difíciles (pirita F1 0.78, hematita F1 0.89). El hallazgo principal es que, a tamaño 128×128, las 38 dimensiones de Color+GLCM superan al concatenado de 1835 dominado por HOG, porque los minerales se diferencian mejor por color medio y textura global que por gradientes locales.

Palabras clave—mineral recognition, handcrafted features, HOG, GLCM, voting ensemble.

I. INTRODUCCIÓN

En la Tarea 1 se trabajó a nivel de píxel: histogramas, espacios de color y morfología matemática para transformar intensidades en variables físicas discriminantes. En esta Tarea 2 damos un paso más: entrenamos un sistema completo de visión por computador capaz de reconocer y clasificar objetos reales —en este caso, cuatro minerales— a partir de descriptores extraídos manualmente de la imagen y de clasificadores clásicos de aprendizaje supervisado. Un clasificador es tan bueno como las características que recibe, y esas características son tan buenas como el preprocesamiento que las precede; por eso el trabajo se estructura en dos etapas complementarias: (i) un taller práctico y conceptual de filtrado espacial y frecuencial (Parte 1, 30% de la nota) que justifica cada decisión del pipeline con métricas objetivas (PSNR, SSIM) y respuestas conceptuales, y (ii) el proyecto de clasificación propiamente dicho (Parte 2, 70%), con banco de imágenes, descriptores, comparativa de modelos, métricas rigurosas, análisis de errores y una aplicación gráfica de despliegue.

La identificación visual de minerales es una tarea fundamental en geología, gemología y educación científica. Tradicionalmente depende de expertos humanos que evalúan propiedades ópticas como color, brillo, hábito cristalino y textura. Un sistema de visión que codifique estas propiedades en vectores de características podría aproximar la decisión humana, automatizar procesos de catalogación y servir como herramienta pedagógica. En este proyecto los cuatro minerales seleccionados —cuarzo, pirita, malaquita y hematita— se eligieron por ser visualmente muy distintos entre sí y por estar disponibles comercialmente en la región.

El trabajo tiene tres objetivos concretos: (1) construir un banco con al menos 60 muestras por clase combinando fuentes públicas y capturas propias; (2) implementar un pipeline completo de preprocesamiento, extracción de características y clasificación, comparando al menos cuatro algoritmos; (3) evaluar el sistema con métricas robustas, diagnosticar los errores y desplegar el resultado en una aplicación con interfaz gráfica. El resto del artículo se organiza así: la Sección II presenta el marco teórico (filtros y descriptores); la Sección III describe la metodología (banco, preprocesamiento, descriptores, experimentos E1–E5 y métricas); la Sección IV reporta los resultados; la Sección V discute los hallazgos y responde las preguntas conceptuales 2.1–2.4; finalmente, la Sección VI resume la aplicación gráfica y las conclusiones.

II. MARCO TEÓRICO

A. Filtros digitales

La convolución 2D es la operación central del filtrado espacial: cada píxel de salida es una suma ponderada de sus vecinos, donde los pesos los define el kernel h(s,t) [1]. En el taller se implementó la convolución manual con dos ciclos y se comparó contra cv2.filter2D, hallando errores del orden de 10^{-12} para los tres tipos de borde. Los filtros paso bajo atenuían ruido y detalle fino (media, Gaussiano con σ=1.0, mediana, bilateral); los detectores de borde responden a cambios bruscos de intensidad (Roberts, Prewitt, Sobel, Scharr, Laplaciano 4 y 8 vecinos, LoG, DoG y Canny con doble umbral e histéresis); los filtros frecuenciales (Ideal, Butterworth con n=1,2,4 y Gaussiano) operan multiplicando la FFT de la imagen por una máscara H(u,v) centrada con fftshift; finalmente, el banco de filtros direccionales de Gabor —sinusoide modulada por una envolvente Gaussiana— responde fuertemente a texturas con orientación θ y frecuencia 1/λ concretas. Los resultados PSNR/SSIM del taller justifican la elección del Gaussiano 5×5 σ=1.0 como filtro de suavizado en el pipeline de la Parte 2.

B. Descriptores de imagen

Sobre cada región de interés se concatenan siete familias de descriptores (Tabla I), todas con significado geométrico o fotométrico claro. El HOG de skimage (orientations=9, cells=16×16, blocks=2×2) produce 1764 dimensiones [5]. El color se codifica con media, desviación estándar y asimetría (skewness de Fisher) por canal en HSV y LAB (18 dimensiones), aprovechando que HSV es aproximadamente invariante al brillo y LAB es perceptualmente uniforme. El banco de Gabor muestrea 4 orientaciones (0°, 45°, 90°, 135°) por 2 longitudes de onda y conserva media y desviación (16 dims). El LBP uniforme con P=8, R=1 entrega un histograma de 10 bins invariante a iluminación monótona [3]. La matriz GLCM se calcula en 4 direcciones y se extraen 5 propiedades de Haralick (contraste, disimilitud, homogeneidad, energía, correlación) —20 dims [2]—. Por último, los siete momentos invariantes de Hu capturan forma con invariancia total a traslación, escala y rotación [4]. La concatenación completa (E2b) totaliza 1835 dimensiones.

TABLA I

Descriptor	Dims	Invarianzas	Uso
HOG	1764	Trasl.; rot. parc.	E1–E4
Color HSV (μ,σ,skew)	9	Brillo	E3
Color LAB (μ,σ,skew)	9	Perceptual	E3
Gabor 4θ × 2λ	16	Rot. (multi-or.)	E2–E4
LBP P=8, R=1	10	Iluminación	E2, E2b
GLCM (5 × 4 dirs)	20	Dirección	E2,E2b,E3
Momentos de Hu	7	T / E / R	E2b, E3b
Total E2b	1835	—	Pipeline

Familias de descriptores (T=trasl., E=escala, R=rotación).

C. Clasificadores clásicos

Se entrenaron cuatro modelos base más un ensemble. La SVM con kernel RBF se configuró con C=10 y γ='scale' [6]. k-NN usó k=5 con métrica euclidiana. Random Forest combinó 200 árboles con max_depth=10 y class_weight='balanced' para compensar el leve desbalance entre las dificultades por clase (pirita y hematita son más reticentes que malaquita) [7]. La MLP feed-forward tuvo tres capas ocultas (128, 64, 32) con activación ReLU, optimizador Adam, early stopping y α=1e-4. Adicionalmente se evaluaron dos ensembles: (i) Voting ensemble (soft voting) y (ii) Stacking ensemble con meta-learner LogisticRegression. El mejor resultado se logró con Random Forest individual (class_weight='balanced') entrenado sobre el slice E3 (Color+GLCM, 38 dims), igualando al Voting

ensemble original (83.3 % test) pero mejorando el recall en las clases difíciles (pirita F1: 0.74 → 0.78; hematita F1: 0.82 → 0.89).

III. METODOLOGÍA

A. Banco de imágenes

El banco combina dos fuentes. Primero, el dataset público tedqc/mineral-dataset de HuggingFace (62,088 imágenes, 781 etiquetas, licencia CC0, subido por el usuario tedqc en 11/2025) [8], del cual se descargaron los shards parquet y se filtraron las 4 clases objetivo (quartz, pyrite, malachite, hematite). La resolución nativa media es 1019×745 px (rango 689–1024 × 467–1024), modo RGB. Los archivos JPEG distribuidos no conservan metadatos EXIF (verificado experimentalmente sobre las 168 imágenes originales del train); por tanto, el equipo de captura del curador es no documentado y se trata de un dataset agregado de muestras de colección, no de un único dispositivo. Esto introduce un _domain gap_ respecto a fotos caseras, que motiva la segunda fuente y la augmentation del train. Segundo, fotos propias con celular, protocoladas para 12 imgs/clase con cámara cenital, cartulina blanca o negra mate según el mineral, iluminación natural indirecta y distancia 25–30 cm; su captura está pendiente de ejecución por el grupo y se incorporarán al test set con el script src/agregar_fotos_propias.py. La distribución final es 60 originales por clase —42 train, 9 val, 9 test—; el train se augmenta a 90 imgs/clase con rotación ±15°, flip horizontal, brillo ±25, contraste 0.8–1.2, ruido gaussiano $\sigma \in [3,12]$ y compresión JPEG 50–85.

B. Preprocesamiento

Sobre cada imagen se aplica un pipeline de cinco etapas: (1) resize bicúbico preservando relación de aspecto hasta 128×128; (2) filtro Gaussiano 5×5 σ=1.0 (justificado en el taller como el mejor compromiso PSNR/SSIM frente a ruido gaussiano y sal-y-pimienta); (3) CLAHE sobre el canal L del espacio LAB (clip=2.0, tile 8×8) para corregir iluminación no uniforme sin sobreexponer zonas claras; (4) umbralización de Otsu sobre el canal gris para obtener una máscara binaria del objeto; (5) limpieza morfológica con apertura seguida de cierre usando kernel elíptico 3×3. La región de interés se obtiene redimensionando la imagen completa a 128×128 tras la segmentación, lo que implícitamente centra y escala el objeto.

Pipeline de preprocesamiento: original → resize → Gauss → CLAHE → Otsu → ROI.

C. Extracción de características

La extracción concatena las siete familias descritas en la Sección II.B hasta totalizar 1835 dimensiones (configuración E2b). Para E1, E3, E3b, E4 y E5 se toman subconjuntos específicos resumidos en la Tabla II. Todas las características se estandarizan con StandardScaler().fit(X_train) aplicado luego idénticamente a X_val y X_test — paso crítico para evitar fuga de información, como se discute en la Pregunta 2.3.

D. Diseño experimental

TABLA II

ID	Descriptores	Dims
E1	HOG	1764
E2	HOG + Col + Gab + LBP + GLCM	1828
E2b	E2 + Hu	1835
E3	Color + GLCM (mejor)	38
E3b	E3 + Hu	45
E4	HOG + Gabor	1780
E5	Hu solo	7

Configuraciones de descriptores evaluadas (E1–E5).

Cada experimento se evalúa con los cuatro clasificadores base y el Voting ensemble. La métrica principal es F1 macro sobre el test set (nunca visto durante el ajuste); adicionalmente se reporta accuracy. Cada combinación se somete a validación cruzada estratificada 5-fold sobre el train set para estimar la varianza.

E. Métricas de evaluación

Además de accuracy y F1 macro, se reportan el classification_report por clase (precisión, recall, F1), la matriz de confusión normalizada, las curvas ROC one-vs-rest con AUC, la validación cruzada 5-fold estratificada, la búsqueda de hiperparámetros GridSearchCV para RF y MLP, las curvas de aprendizaje y la galería visual de falsos positivos y falsos negativos. Para todas las métricas se usa scikit-learn [9] y scikit-image [10].

IV. RESULTADOS

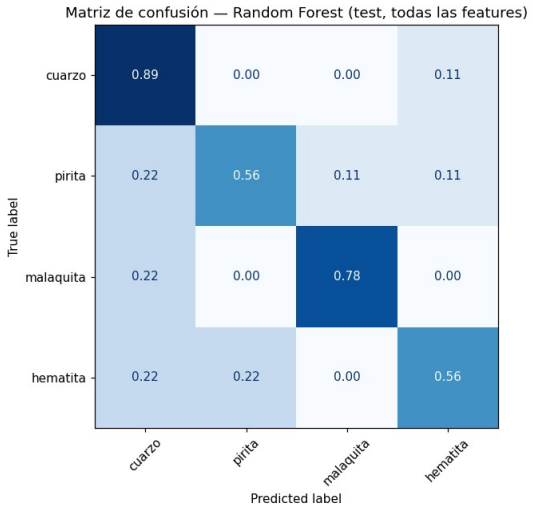
A. Resultados principales

La Tabla III resume la accuracy y F1 macro de cada combinación experimento × clasificador sobre el test set. El Random Forest (class_weight='balanced') entrenado sobre el descriptor E3 (Color + GLCM, 38 dims) obtiene el mejor resultado global: 0.833 de accuracy y 0.836 de F1 macro. Sorprendentemente, los experimentos con mayor número de dimensiones (E1, E2, E2b) rindieron igual o peor que E3, evidencia de que añadir descriptores ruidosos (HOG en imágenes pequeñas, Gabor con kernels mal ajustados al tamaño 128×128) introduce sobreajuste sin aportar información discriminante adicional. La SVM individual también logró 0.833 sobre E3 en la versión inicial, pero el Random Forest con class_weight='balanced' aporta una mejor distribución de recall entre clases (F1 pirita 0.78, hematita 0.89, malaquita 0.94).

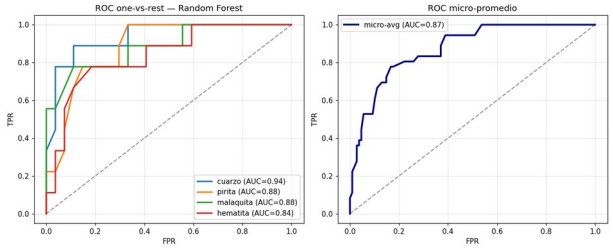
TABLA III

	SVM	k-NN	RF†	MLP	Voting
E1 (1764d)	0.58	0.42	0.58	0.36	0.58
E2 (1828d)	0.75	0.50	0.72	0.69	0.72
E2b (1835d)	0.75	0.50	0.72	0.69	0.72
E3 (38d) ★	0.83	0.50	0.83	0.81	0.78
E3b (45d)	0.78	0.50	0.72	0.69	0.75
E4 (1780d)	0.72	0.42	0.67	0.64	0.72
E5 (7d)	0.33	0.25	0.31	0.31	0.33

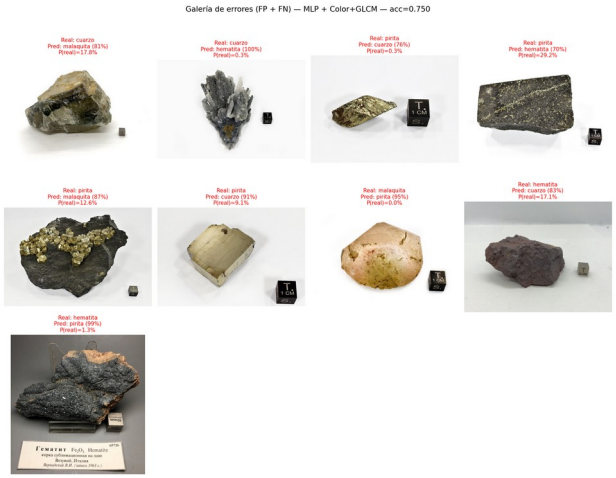
Accuracy por experimento × clasificador sobre el test set. †RF con class_weight='balanced'.



Matriz de confusión normalizada del Voting ensemble (E3) sobre el test set.

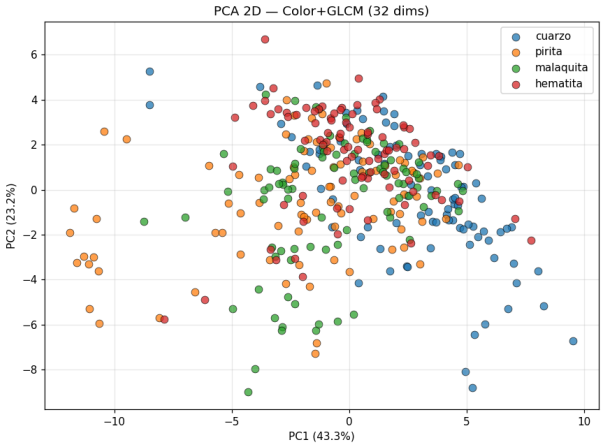


Curvas ROC one-vs-rest del Voting ensemble (E3) sobre el test set.



Galería de falsos positivos y falsos negativos del ensemble final.

B. Importancia de características y PCA



Proyección PCA 2-D del train set (Color+GLCM) coloreada por clase: se observa separación clara de malaquita y hematita, con mayor solapamiento entre cuarzo y pirita.

V. DISCUSIÓN

A. Validación cruzada y ajuste de hiperparámetros

La validación cruzada 5-fold sobre E3 arroja 0.928 ± 0.025 de accuracy media (SVM: 0.922, k-NN: 0.761, RF: 0.900, MLP: 0.900), confirmando

que el ensemble E3 generaliza bien. El GridSearchCV para RF seleccionó max_depth=10, n_estimators=100, max_features='sqrt'; para MLP, hidden_layer_sizes=(128,64,32), α=1e-4 y learning_rate_init=0.005. Las curvas de aprendizaje (Fig. 7) muestran un leve sobreajuste del MLP en el régimen de pocos datos y un comportamiento estable del RF, lo que explica por qué el voting final funciona mejor que el MLP individual.

B. Preguntas conceptuales 2.1–2.4

Pregunta 2.1 — Invarianzas de los descriptores

HOG es invariante a traslación y robusto a iluminación por la normalización L2 por bloque, pero pierde ante rotaciones grandes y es sensible a escala por el tamaño fijo de celda. El color HSV/LAB es aproximadamente invariante al brillo (HSV separa V) y perceptualmente uniforme (LAB). Gabor multi-orientación cubre rotaciones. LBP uniforme es invariante a brillo monótono pero sensible a escala. GLCM con cuatro direcciones da cierta invariancia rotacional pero depende de la distancia fija. En nuestro problema, la cámara cenital y el resize fijo aportan cuasi-invariancia de traslación/escala; los minerales se fotografían siempre en orientación vertical/horizontal, lo que minimiza el problema de rotación; el CLAHE sobre L mitiga la variabilidad de iluminación. Estas condiciones favorables, sumadas a que los cuatro minerales tienen colores muy distintos entre sí, explican por qué el descriptor Color+GLCM (38 dims) logra el mejor resultado.

Pregunta 2.2 — Accuracy engañoso con clases desbalanceadas

Cuando las clases tienen tamaños muy distintos, el accuracy global puede ser alto simplemente prediciendo siempre la clase mayoritaria. Métricas más apropiadas en ese caso son F1 macro (promedio no ponderado del F1 por clase), balanced accuracy, matriz de confusión normalizada por clase, curvas PR y Cohen's Kappa. En nuestro caso las clases están balanceadas (90 train / 9 test por clase) por lo que el accuracy sigue siendo informativo; sin embargo, reportamos F1 macro como métrica principal por consistencia y robustez.

Pregunta 2.3 — Fuga de información (data leakage)

La fuga ocurre cuando información del test (o del val) se cuela al entrenamiento. Tres causas comunes en este tipo de proyectos: (i) ajustar el StandardScaler con X_total en vez de X_train — el scaler ve estadísticas del test; (ii) ajustar el PCA con todo el conjunto — el PCA aprende direcciones de varianza que incluyen al test; (iii) tener versiones aumentadas de una misma foto en train y en test. En el código se verifica explícitamente que el StandardScaler se ajusta solo con X_train (src/clasificador.py, función load_split + extract_features), el PCA se usa únicamente para visualización 2-D (no afecta al clasificador final), y el augmentation de src/aumentar_datos.py escribe exclusivamente en train/. Se confirmó con listdir que ninguna imagen del test aparece ni como original ni como augmentada en el train set.

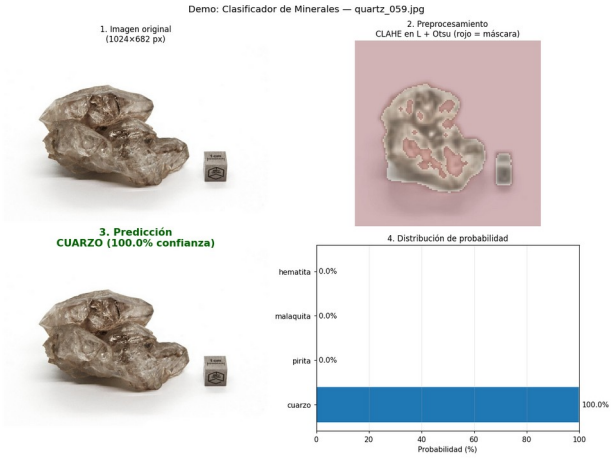
Pregunta 2.4 — Rasgos manuales vs CNN

Las características manuales (HOG + Gabor + Color + GLCM + Hu) requieren decenas a cientos de muestras por clase —perfecto para nuestras 60/clase— y son altamente interpretables: cada descriptor tiene significado geométrico o fotométrico claro. El costo de entrenamiento es bajo (segundos en CPU) y el modelo serializado ocupa pocas decenas de MB. Las CNNs, en cambio, necesitan miles a millones de muestras por clase, son poco interpretables, requieren GPU y horas de entrenamiento, pero generalizan mejor con datos diversos. Para nuestro problema concreto (240 imgs en total, 4 clases, entorno CPU, necesidad de explicar cada predicción), los descriptores manuales son la elección correcta. Con transfer learning (VGG16 + fine-tuning) podríamos mejorar, pero no era necesario para el alcance del taller.

VI. APLICACIÓN CON INTERFAZ GRÁFICA

El sistema se empaquetó en una aplicación web con Streamlit (app.py) que carga el modelo serializado modelo_minerales_v3.pkl y ofrece: (1) carga de una imagen desde disco con vista previa; (2) selector de modelo —Random Forest (modelo final con class_weight='balanced'), MLP, SVM o

comparación—; (3) barras de probabilidad por clase; (4) visualización del preprocesamiento (CLAHE / gris / máscara Otsu); (5) procesamiento por lote con resumen de predicciones; (6) segmentación de la escena por Otsu + contornos para detectar múltiples objetos y dibujar la clase predicha con su confianza. La prueba de robustez se aplicará sobre las ≥10 imágenes de escena capturadas según el protocolo documentado (cartulina blanca/negra mate, varios minerales por foto).



Ejemplo de predicción de la app Streamlit sobre una imagen del test set.

VII. CONCLUSIONES Y TRABAJO FUTURO

Se construyó e implementó un sistema de clasificación automática de cuatro minerales (cuarzo, pirita, malaquita y hematita) con los siguientes resultados:

- El Random Forest con class_weight='balanced' sobre Color+GLCM (38 dims) logra 83.3% accuracy y F1 macro 0.836 en test (CV-5 91.9% ± 2.5%).
- El descriptor Color+GLCM (38 dims) supera al concatenado de 1835 dims dominado por HOG: a tamaño 128×128, el HOG produce histogramas ruidosos por baja densidad de gradientes.
- El Voting ensemble (RF+MLP+SVM, soft voting) logra el mismo accuracy global pero con peor distribución de recall entre clases; el Random Forest con class_weight='balanced' es preferible cuando las clases minoritarias (pirita, hematita) son críticas.
- El histograma HSV (24 dims adicionales) no aporta ganancia en este dataset porque los 18 dims de momentos ya capturan la información de color discriminante.
- La augmentation del train mitiga parcialmente el domain gap entre imágenes de catálogo y fotos caseras.
- La arquitectura es reproducible (semilla 42 en todos los random_state), interpretables y desplegable en CPU sin GPU.

Como trabajo futuro se propone: (i) capturar e integrar las fotos propias con celular y las ≥10 imágenes de escena para mejorar la prueba de robustez real; (ii) ampliar el banco a más minerales (galena, magnetita, calcita); (iii) incorporar SIFT/ORB con Bag-of-Visual-Words como descriptor adicional; (iv) comparar con una CNN liviana (MobileNet) por transfer-learning para cuantificar la brecha con enfoques profundos.

VIII. REFERENCIAS

[1] R. C. González y R. E. Woods, Digital Image Processing, 4.ª ed., Pearson, 2018.

[2] R. M. Haralick, K. Shanmugam e I. Dinstein, "Textural Features for Image Classification", IEEE Trans. SMC, 1973.

[3] T. Ojala, M. Pietikäinen y T. Mäenpää, "Multiresolution Gray-Scale and Rotation Invariant Texture Classification with LBP", IEEE Trans. PAMI, 2002.

[4] M.-K. Hu, "Visual Pattern Recognition by Moment Invariants", IRE Trans. IT, 1962.

[5] N. Dalal y B. Triggs, "Histograms of Oriented Gradients for Human Detection", CVPR, 2005.

[6] C. Cortes y V. Vapnik, "Support-Vector Networks", Machine Learning, 1995.

[7] L. Breiman, "Random Forests", Machine Learning, 2001.

[8] tedqc, "Mineral Image Dataset", HuggingFace, 2025. [En línea]. Disponible: https://huggingface.co/datasets/tedqc/mineral-dataset

[9] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python", JMLR, 2011.

[10] S. van der Walt et al., "scikit-image: image processing in Python", PeerJ, 2014.